

Guide to the Linear Regression Colab Exercises

Companion to `Exercises_LinReg.ipynb`

How to use this guide. Read the *key material* section before starting. Then, for each question, try on your own first. If you are stuck for more than a few minutes, open the hints in order: **Hint 1** points you in the right direction, **Hint 2** gives the method, **Hint 3** takes you to the last step. The hints never give the final numbers: the point is that you produce them. The *common mistakes* boxes list what goes wrong most often; read them even when you think you are done.

Key material

The model and its loss in matrix form

With n observations and p features, build the design matrix $A = [\mathbf{1} \ X]$ of shape $n \times (p + 1)$ (a column of ones for the intercept). The parameters are $\theta = (b, w_1, \dots, w_p)^\top$, the predictions $\hat{y} = A\theta$, and the loss

$$L(\theta) = \frac{1}{n} \|y - A\theta\|^2, \quad \nabla L(\theta) = -\frac{2}{n} A^\top (y - A\theta).$$

Setting the gradient to zero gives the **normal equations** $A^\top A \theta = A^\top y$.

Gradient descent

Start from $\theta^{(0)}$ and repeat $\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)})$. The learning rate η must be small enough for the *most curved* direction of L , otherwise the iterates oscillate and explode. The speed along the *least curved* direction is then limited by that same η . The ratio of largest to smallest curvature is the **condition number** of $A^\top A$: the larger it is, the slower gradient descent converges. Standardising features makes the curvature similar in all directions.

Evaluation metrics

RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$, in the units of y : a typical error size.
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$: fraction of variance explained. 0 = no better than predicting the mean; it can be negative on new data.
Residual plot	residuals against a feature or against $\hat{y}$. A good model leaves a shapeless cloud around 0; any pattern (curve, funnel) means the model is missing something.

scikit-learn, CV scoring names are "higher is better", so errors come negated: "neg_root_mean_squared_error". Put a minus sign in front of the result.

Mathematical insight · least squares as a projection

The fitted vector $\hat{y} = A\hat{\theta}$ is the point of the column space of A closest to y . The normal equations say $A^\top (y - A\hat{\theta}) = 0$: the residual vector is *orthogonal to every column* of A . Consequences:

- with an intercept column of ones, $\mathbf{1}^\top e = 0$: residuals sum to zero;
- $\hat{y} = Hy$ with the *hat matrix* $H = A(A^\top A)^{-1}A^\top$, a projection ($H^2 = H$) whose trace is the number of parameters;

- Pythagoras: $\|y - \bar{y}\mathbf{1}\|^2 = \|\hat{y} - \bar{y}\mathbf{1}\|^2 + \|e\|^2$, i.e. $SST = SSR + SSE$, which is why $0 \leq R^2 \leq 1$ on the training data.

Mathematical insight · the statistical view

If $y = A\theta + \varepsilon$ with independent Gaussian noise of variance σ^2 , least squares is the maximum-likelihood estimator, it is unbiased, and

$$\text{Cov}(\hat{\theta}) = \sigma^2(A^\top A)^{-1}.$$

Small eigenvalues of $A^\top A$ (nearly collinear columns) mean large variances of the coefficients. For feature j , the variance is inflated by the *variance inflation factor* $\text{VIF}_j = 1/(1 - R_j^2)$, where R_j^2 is the R^2 of regressing feature j on the other features. A correlation of 0.86 between two features alone gives a VIF of about $1/(1 - 0.74) \approx 3.8$.

Exercise 1 Gradient descent from scratch

1.1 fit_normal_equations

Hint 1 · Nudge Two lines: build A with `add_ones`, then solve a linear system.

Hint 2 · Method `np.linalg.solve(M, v)` solves $M\theta = v$. Here $M = A^\top A$ ($A.T @ A$) and $v = A^\top y$.

Hint 3 · Almost there Why not `np.linalg.inv`? Solving is faster and numerically more accurate than inverting and then multiplying.

Mathematical insight · convergence of gradient descent on a quadratic

Let $M = \frac{2}{\eta}A^\top A$ with eigenvalues $\lambda_{\min} \leq \dots \leq \lambda_{\max}$. The error $\theta^{(t)} - \hat{\theta}$ evolves as $(I - \eta M)^t(\theta^{(0)} - \hat{\theta})$. Along an eigenvector with eigenvalue λ , it is multiplied by $(1 - \eta\lambda)$ at each step. Hence:

- the iteration **diverges** if $\eta > 2/\lambda_{\max}$ (some factor has absolute value above 1): this is your `lr=1e-4` run;
- the slowest direction shrinks by $(1 - \eta\lambda_{\min})$; with the best $\eta = 2/(\lambda_{\min} + \lambda_{\max})$ the rate is $\frac{\kappa-1}{\kappa+1}$ per step, where $\kappa = \lambda_{\max}/\lambda_{\min}$ is the condition number;
- the number of steps to gain a factor of 10 grows roughly like κ . With $\kappa \approx 3 \times 10^5$, thousands of steps barely move the flat directions; with $\kappa \approx 1$, a handful of steps suffice.

1.2 gradient_descent

Hint 1 · Nudge Initialise `theta = np.zeros(A.shape[1])` and an empty list. Inside the loop: residuals, loss, gradient, update.

Hint 2 · Method Residual vector `r = y - A @ theta`; loss `np.mean(r ** 2)`; the gradient is `-(2 / n) * A.T @ r`. Append the loss *before* updating `theta`.

Hint 3 · Almost there Then two calls: one with `X`, `lr=3e-5`; one with `Xz`, `lr=0.1`; both with `n_iter=3000`. Plot `np.array(hist) - mse_opt` with `plt.semilogy`.

Common mistakes

- Sign error in the update: with the gradient written as $-\frac{2}{n}A^T r$, the update *subtracts* it, which amounts to *adding* $\eta \frac{2}{n}A^T r$.
- Forgetting the $1/n$: the effective learning rate becomes n times larger and the run diverges.
- Standardising the column of ones: the intercept column must stay equal to 1. Standardise `X` first, then call `add_ones`.

1.3 Back to original units

Hint 1 · Nudge Write the standardised model and expand it: $\hat{y} = \tilde{b} + \sum_j \tilde{w}_j \frac{x_j - \mu_j}{s_j}$.

Hint 2 · Method Collect the terms that multiply x_j : they give w_j . Collect the terms that do not depend on any x_j : they give b .

Hint 3 · Almost there $w_j = \tilde{w}_j/s_j$ and $b = \tilde{b} - \sum_j \tilde{w}_j \mu_j/s_j$. In NumPy, `theta_z[0]` is $\tilde{b}$ and `theta_z[1:]` is the vector $\tilde{w}$; `mu` and `sd` are vectors of the same length.

Question 1 (written)

Hint 1 · Nudge Use three pieces of evidence from your outputs: the gap after 3000 iterations for both runs, the diverging run with `lr=1e-4`, and the two condition numbers.

Hint 2 · Method Which feature has the largest scale? It makes the loss steepest in its direction. Which parameters are “flat” directions? Compare `theta_raw` with `theta_ne` entry by entry.

Hint 3 · Almost there Explain why increasing the learning rate does not help: what happens to the steep direction when η grows?

Exercise 2 Nucleus area from radius

Feature engineering keeps the model linear

“Linear regression” means linear **in the parameters**, not in the inputs. Any fixed transformation of the inputs can be used as a feature: $\hat{y} = w_1x + w_2x^2$ is still linear regression, fitted with exactly the same machinery. scikit-learn tools:

```
PolynomialFeatures(2, include_bias=False) # x -> [x, x^2]
FunctionTransformer(np.log)               # x -> log(x); any function works
LinearRegression(fit_intercept=False)     # forces the line through 0
```

Mathematical insight · why a high R^2 can hide a wrong model

R^2 compares the model with the constant $\bar{y}$. When y varies a lot over the range of x (area goes from about 150 to 2500), almost any increasing curve explains most of that variation, so R^2 is high even if the shape is wrong. Residual plots test a different thing: whether the errors are *systematic*. If $\mathbb{E}[y \mid x]$ is not linear, the residuals of a straight-line fit estimate the missing part $\mathbb{E}[y \mid x] - (wx + b)$, which is a smooth curve, not noise.

2.2 The straight line

Hint 2 · Method Fit on `r_train`, `a_train`; `.score` gives R^2 . Residuals: `a_train - lin.predict(r_train)`; scatter them against `r_train` and add `plt.axhline(0)`.

Hint 3 · Almost there `r` is already 2-D (shape `(n, 1)`), which scikit-learn needs; `a` is 1-D. Do not flatten `r`.

Question 2 (written)

Hint 1 · Nudge Is a high R^2 enough to call a model good? What does the residual plot look like?

Hint 2 · Method If residuals are positive at both ends and negative in the middle, what shape is the truth relative to the line?

Hint 3 · Almost there Which geometric formula links a disc's area to its radius?

2.3 Three models

Hint 2 · Method Put the three models in a dict. Model B: `make_pipeline(PolynomialFeatures(...), LinearRegression())`. Model C: `make_pipeline(FunctionTransformer(np.square), LinearRegression(fit_intercept=False))`.

Hint 3 · Almost there A dict comprehension computes all three scores at once: `{name: -cross_val_score(m, ...).mean() for name, m in models.items()}`. To read C's coefficient, the last step of a pipeline is `model_c[-1]`.

2.4 Test once

Hint 2 · Method `mean_squared_error(a_test, pred) ** 0.5` for the RMSE and `r2_score(a_test, pred)`.

Question 3 (written)

Hint 1 · Nudge Compare the CV RMSE of B and C. Is the difference meaningful, given that each is an average of 5 noisy folds?

Hint 2 · Method If two models perform the same, which do you prefer: the one with fewer parameters and a physical meaning, or the other?

Hint 3 · Almost there Compare C 's coefficient with a famous constant. Why is it not exactly equal? (Nuclei are not perfect discs, and each value is an average over many nuclei.)

Exercise 3 Alcohol content of wines

Baselines and honest comparisons

Always compare with a model that ignores the features: `DummyRegressor()` predicts the training mean, so its R^2 is about 0. A model is only interesting if it clearly beats it. When comparing two real models, look at the difference in mean CV score *and* at the fold-to-fold standard deviation: a difference much smaller than that spread is not evidence.

Correlated features and unstable coefficients

When two features are strongly correlated, the data cannot tell how to share the effect between them: many combinations of their coefficients fit almost equally well. Small changes in the data then move the coefficients a lot, sometimes changing their sign. A coefficient w_j means “the change in $\hat{y}$ when x_j increases by one unit *with all other features held fixed*”, which can differ, even in sign, from the simple correlation between x_j and y . The **bootstrap** measures this instability: resample the training rows with replacement many times, refit, and look at the spread of each coefficient.

3.2 Four models

Hint 2 · Method A dict of the four models (three of them are pipelines with `StandardScaler()` first), one `cross_val_score(..., scoring="r2")` each, and one dictionary per model in a list of rows.

Hint 3 · Almost there `pd.DataFrame(rows)` with keys `model`, `mean_r2`, `std_r2` gives the table in the right column order.

Mathematical insight · why the bootstrap works, and Ridge in the eigenbasis

The bootstrap treats the training set as a stand-in for the population: resampling it with replacement mimics drawing new training sets, so the spread of the refitted coefficients approximates their sampling variability without any formula.

Ridge acts directly on this variability. Writing the SVD of the (centred, scaled) design matrix $A = UDV^\top$, least squares fits $\hat{y} = \sum_j u_j u_j^\top y$, while Ridge fits

$$\hat{y}_\lambda = \sum_j \frac{d_j^2}{d_j^2 + \lambda} u_j u_j^\top y.$$

Directions with large singular values d_j are barely touched; directions with small d_j (the ones created by correlated features, where coefficients are unstable) are shrunk strongly. Ridge trades a little bias for a large reduction of variance exactly where the bootstrap shows instability.

3.3 Bootstrap

Hint 1 · Nudge Convert to NumPy first (`.to_numpy()`) so that indexing with an integer array selects rows.

Hint 2 · Method In each of the 200 iterations: `idx = rng.integers(0, n, n)` (indices with repetition), fit the scaled pipeline on the rows `idx`, append `pipeline[-1].coef_`.

Hint 3 · Almost there `boot = np.array(list_of_coefs)` has shape `(200, 12)`. The fraction of positive values per feature is `(boot > 0).mean(axis=0)`; wrap it in `pd.Series(..., index=Xw.columns)`.

3.4 Test once

Hint 2 · Method Refit your chosen model on the whole training set and call `.score(Xw_test, yw_test)`. Do the same for a `DummyRegressor()`.

Question 4 (written)

Hint 1 · Nudge (a) From your CV table, is any model clearly better than the others once you take the standard deviations into account? What tie-breaking principle did you use in the k -NN notebook?

Hint 2 · Method (b) Sort the features by their fraction of positive fits. Which ones are (nearly) always the same sign? Which ones hover around 0.5 to 0.8?

Hint 3 · Almost there Match the unstable features with the most correlated pairs printed in 3.1. For at least one feature, compare the sign of its coefficient with the sign of its correlation with alcohol (`Xw_train.join(yw_train).corr()`), and explain any disagreement.

Check yourself before moving on

- Why does gradient descent need scaled features while the normal equations do not?
- Why can a model with $R^2 = 0.97$ still be the wrong model?
- What does “linear” refer to in “linear regression”?
- When may you interpret the sign of a regression coefficient, and when not?